

QUIZ: UDP vs TCP

A server handling 10 clients simultaneously. How many socket connections are maintained by **the server** if the server and client use UDP?

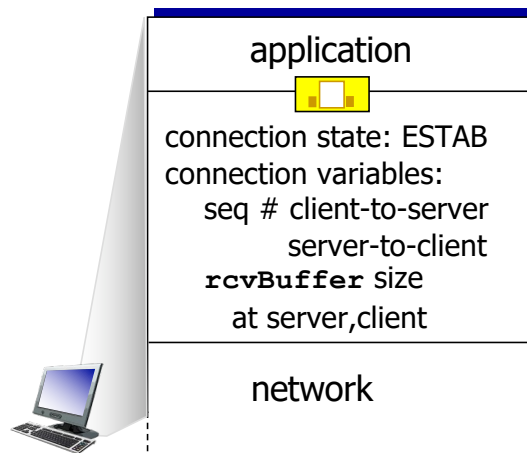
And how many socket connections maintained by the server if they use TCP?

Answer: 1 socket connection when UDP is used, and $10 + 1 = 11$ socket connections when TCP is used.

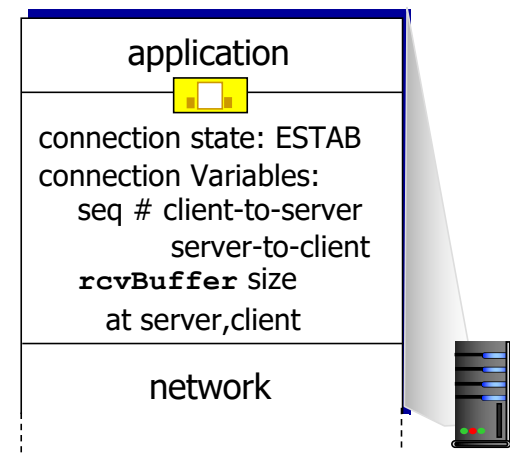
TCP connection management

before exchanging data, sender/receiver “handshake”:

- agree to establish connection (each knowing the other willing to establish connection)
- agree on connection parameters (e.g., starting seq #s)



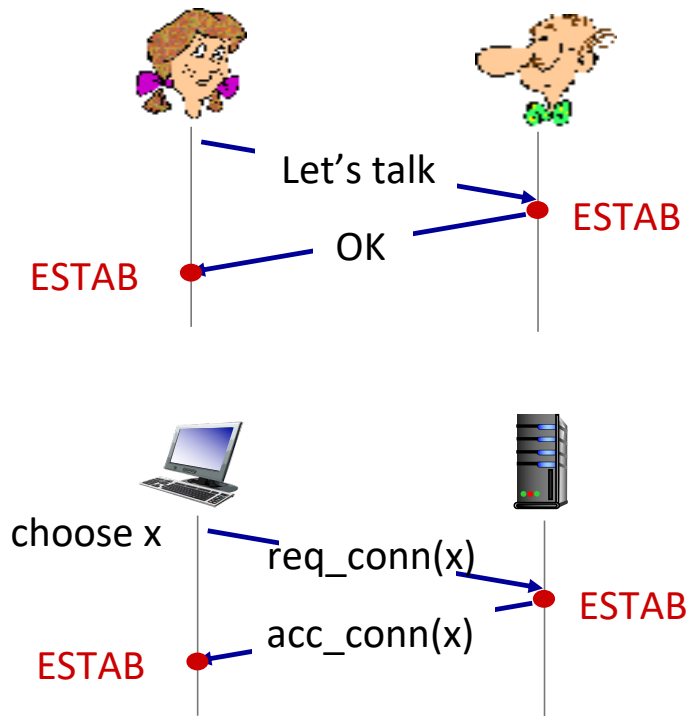
```
Socket clientSocket =  
    newSocket("hostname", "port number");
```



```
Socket connectionSocket =  
    welcomeSocket.accept();
```

Agreeing to establish a connection

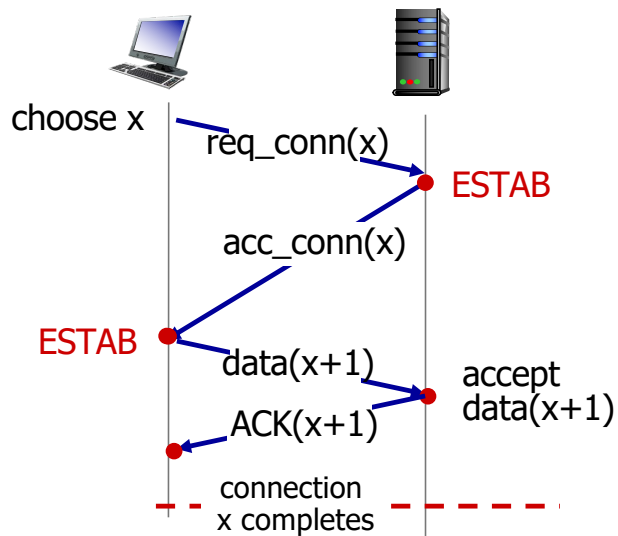
2-way handshake:



Q: will 2-way handshake always work in network?

- variable delays
- retransmitted messages (e.g. req_conn(x)) due to message loss
- message reordering
- can't "see" other side

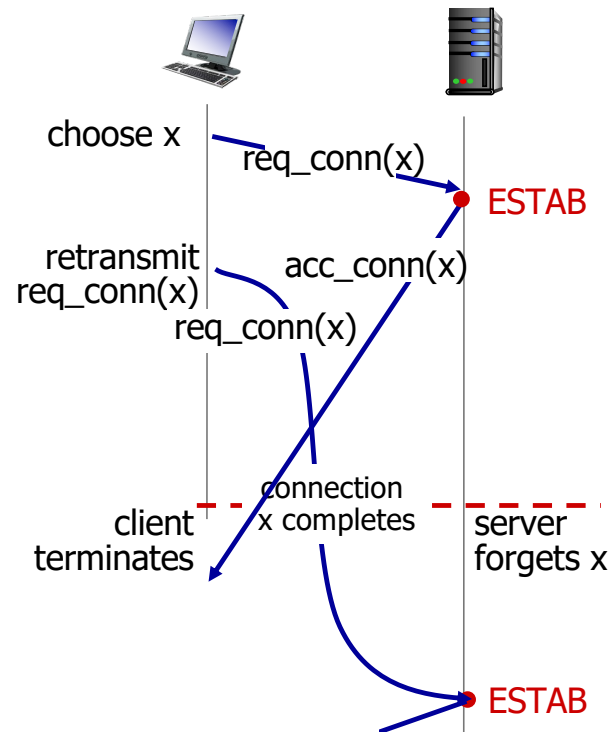
2-way handshake scenarios



No problem!



2-way handshake scenarios



Problem: half open connection! (no client)

TCP 3-way handshake

Client state (A)

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName, serverPort))
```

SYNSENT

ESTAB

choose init seq num, x
send TCP SYN msg

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data



SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

ACKbit=1, ACKnum=y+1



choose init seq num, y
send TCP SYNACK
msg, acking SYN

received ACK(y)
indicates client is live

Server state (B)

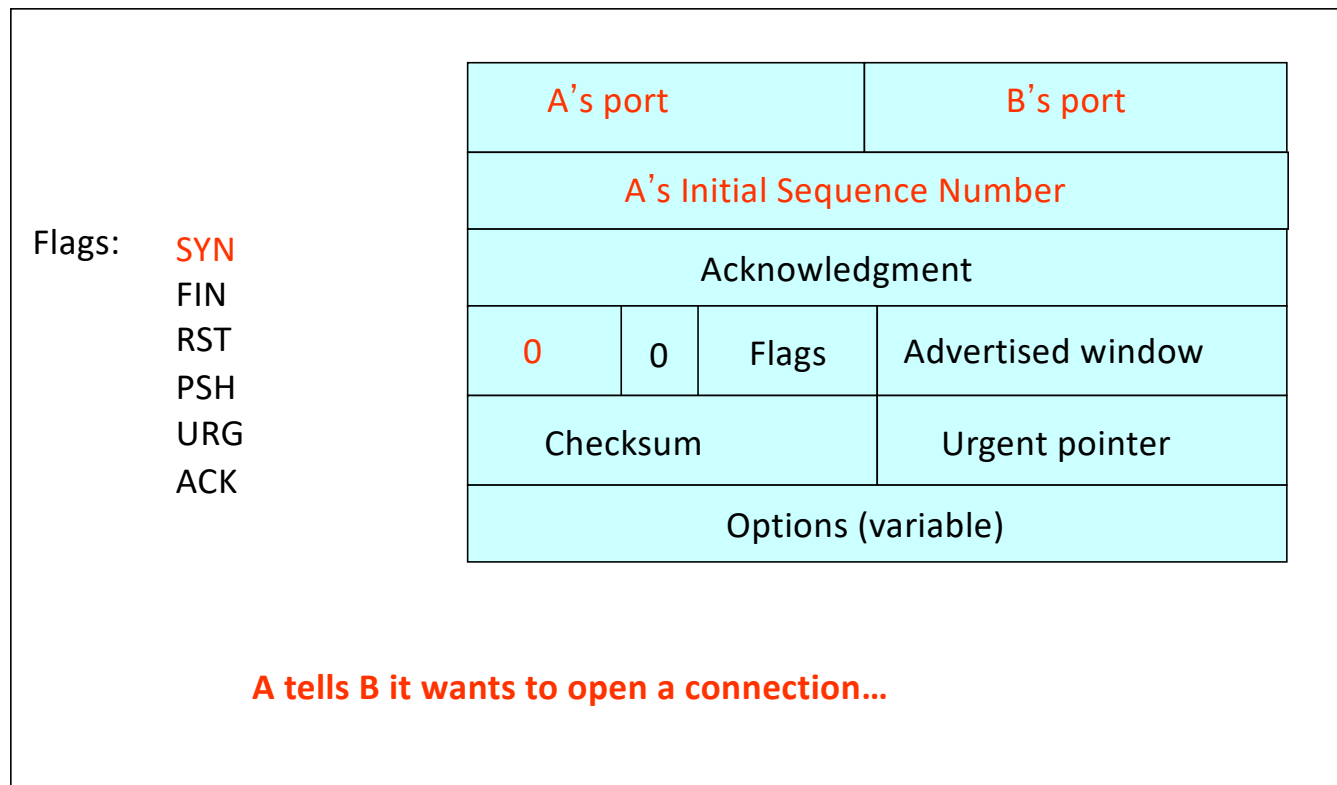
```
serverSocket = socket(AF_INET, SOCK_STREAM)  
serverSocket.bind(('', serverPort))  
serverSocket.listen(1)  
connectionSocket, addr = serverSocket.accept()
```

LISTEN

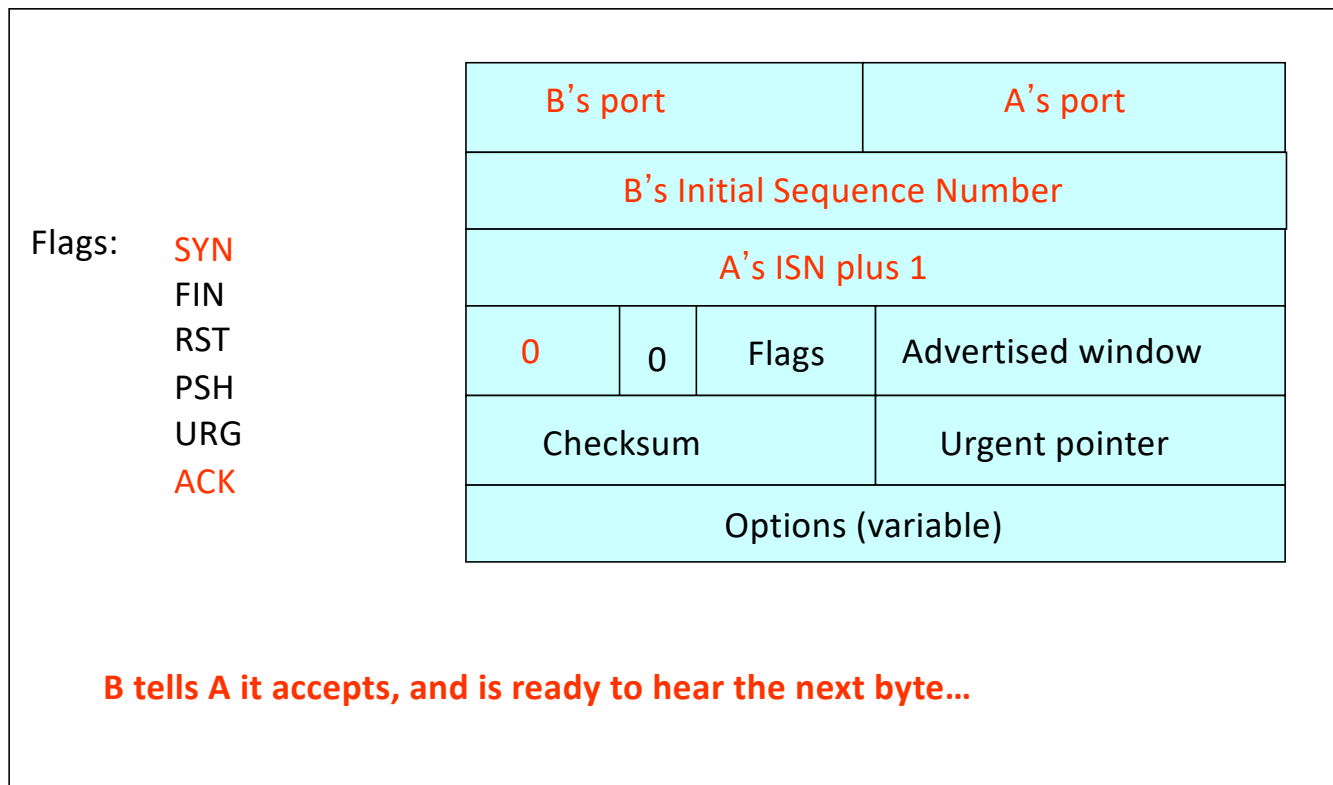
SYN RCVD

ESTAB

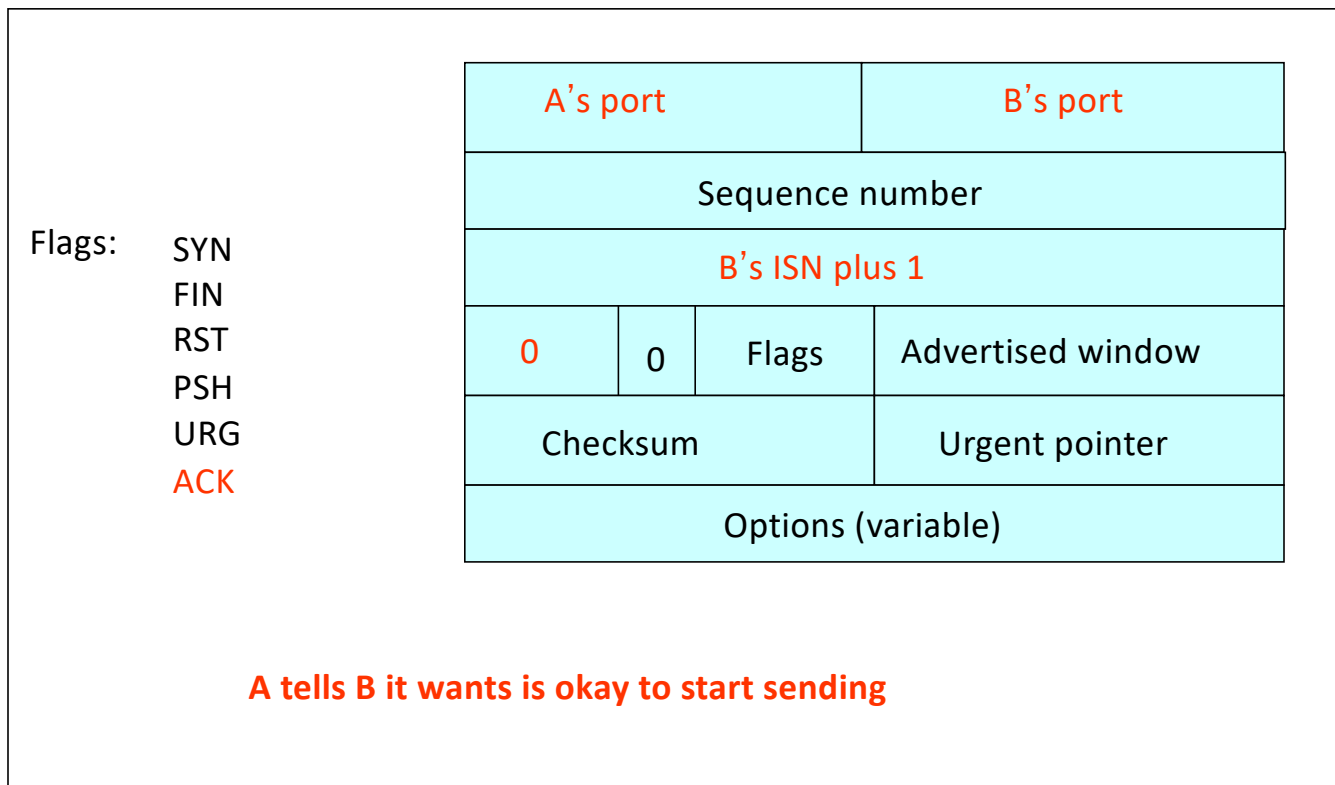
Step 1: A's initial SYN packet



Step 2: B's SYN-ACK packet



Step 3: A's ACK of the SYN-ACK

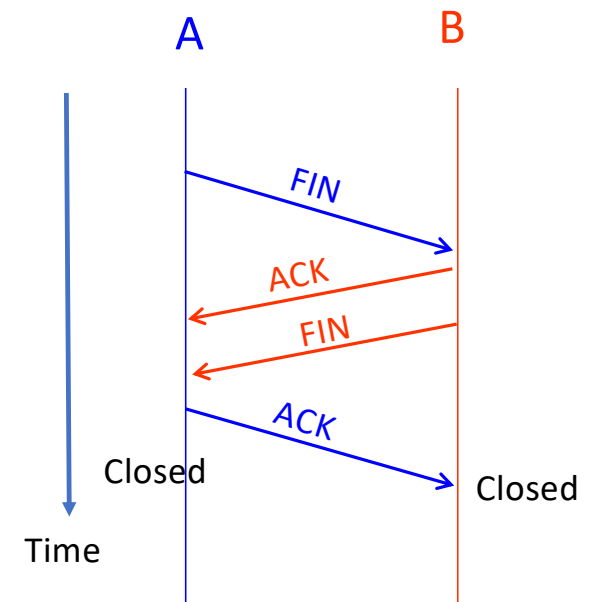


A human 3-way handshake protocol



Closing a TCP connection

- client, server each close their side of connection
 - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
 - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled



What does TCP do for reliable data transfer?

- GBN + selective repeat + a few differences
 - Sequence number, sliding window at TX and RX
 - RX sends cumulative ACKs (GBN)
 - TX maintains a single retransmission timer
 - RX keeps out-of-order packets (like SR)
 - Use **fast retransmit**: use duplicate ACKs to trigger early retransmission
 - **Estimate timeout**: retransmit if no ACK is received after timeout

What can go wrong in a data transmission?

- Bit errors

Checksum, ACK

- Duplicated packets

Sequence numbers, estimate RTT to set timeout

- Packet losses

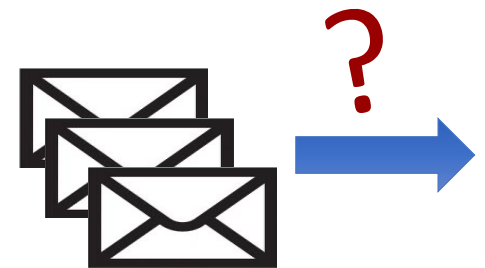
Timer, duplicated ACKs,

- Packet delay

Fast retransmit

- Packets arriving out of order

Sequence numbers, keep out-of-order packets, ACK



Transport layer: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP

Basics and UDP

- Principles of reliable data transfer
- Connection-oriented transport: TCP

Reliability

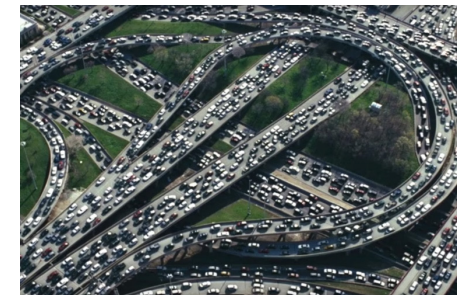
- **Principles of congestion control**
- TCP congestion control
- Evolution of transport-layer functionality

Congestion Control

Principles of congestion control

Congestion:

- informally: “too many sources sending too much data too fast for *network* to handle”
- manifestations:
 - long delays (queueing in router buffers)
 - packet loss (buffer overflow at routers)
- different from flow control!
- a top-10 problem!



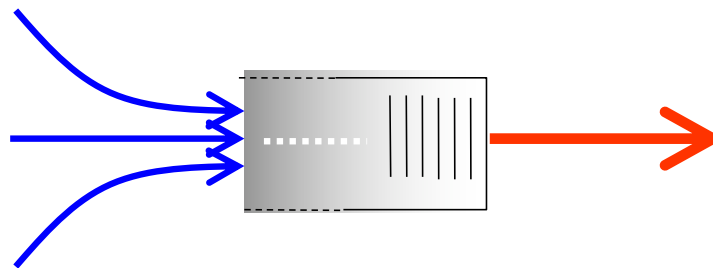
congestion control:

too many senders,
sending too fast

flow control: one sender
too fast for one receiver

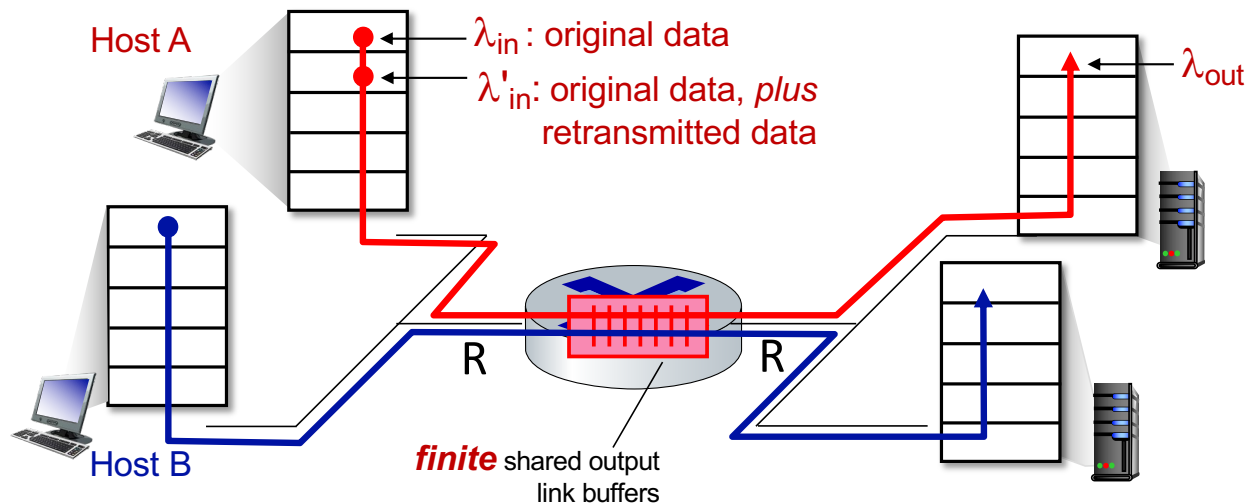
Why Does Congestion Happen?

- Statistical multiplexing in packet switching
- If too many packets arrive at a router in a sudden
 - The router cannot keep up with the traffic
 - Delay traffic, and the buffer may eventually overflow



Causes/costs of congestion: scenario 1

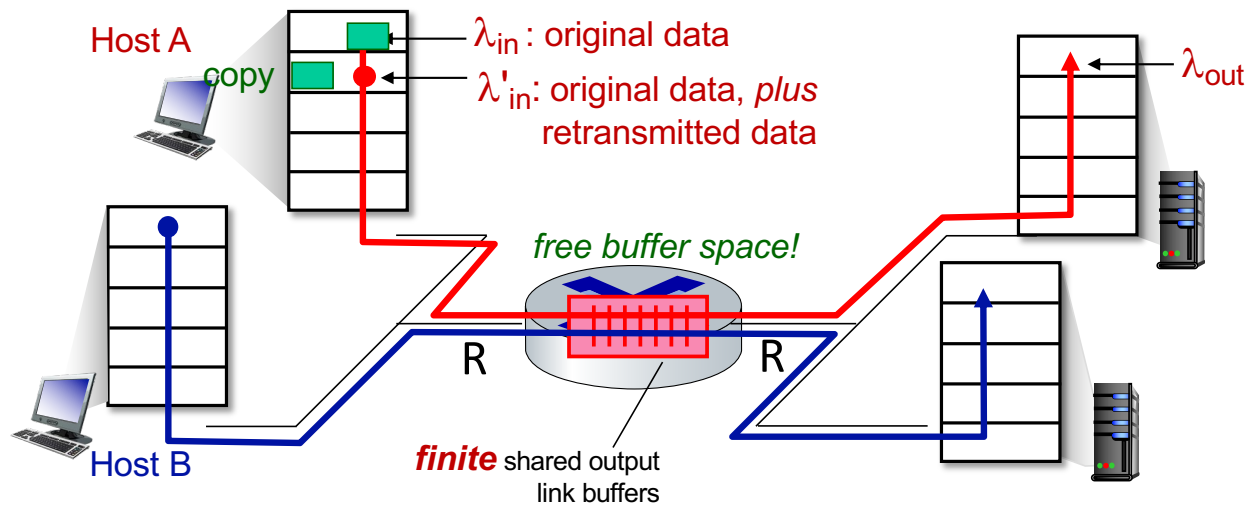
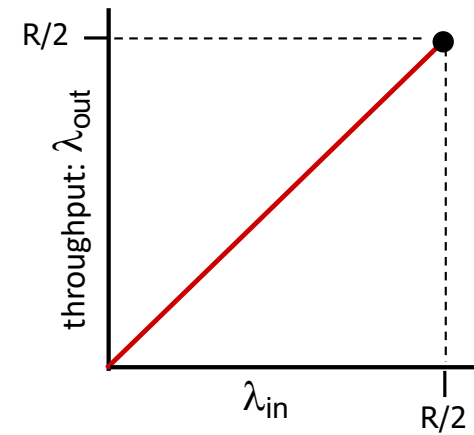
- one router, *finite* buffers
- sender retransmits lost, timed-out packet
 - application-layer input = application-layer output: $\lambda_{in} = \lambda_{out}$
 - transport-layer input includes *retransmissions* : $\lambda'_{in} \geq \lambda_{in}$



Causes/costs of congestion: scenario 1

Idealization: **perfect knowledge**

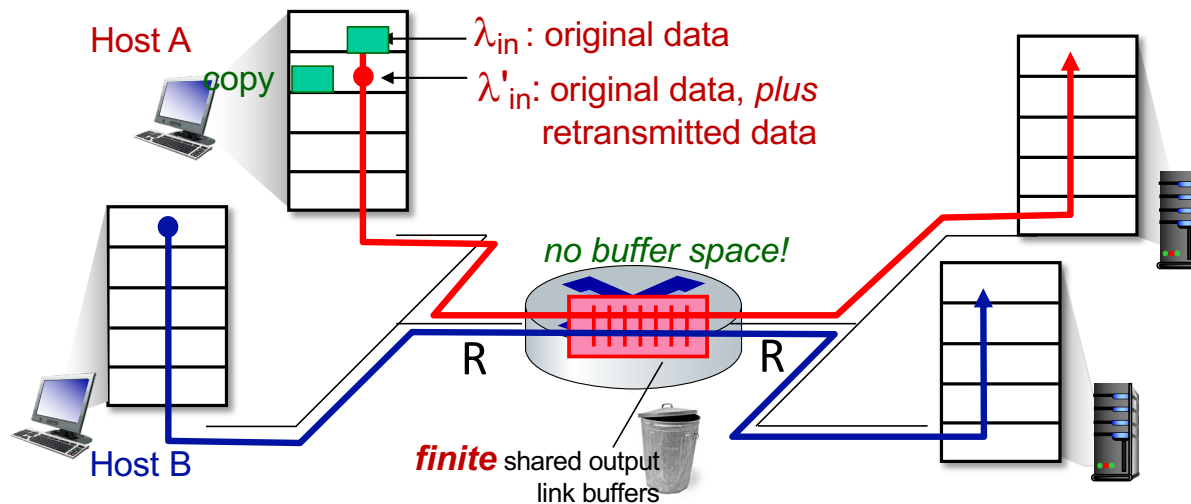
- sender sends only when router buffers available



Causes/costs of congestion: scenario 1

Idealization: *some* perfect knowledge

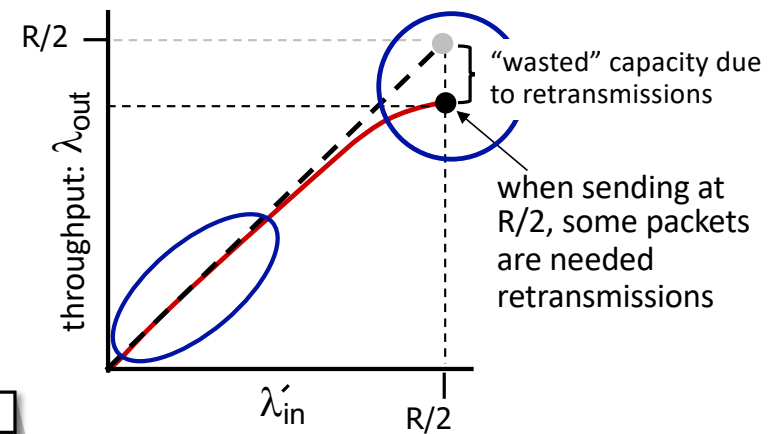
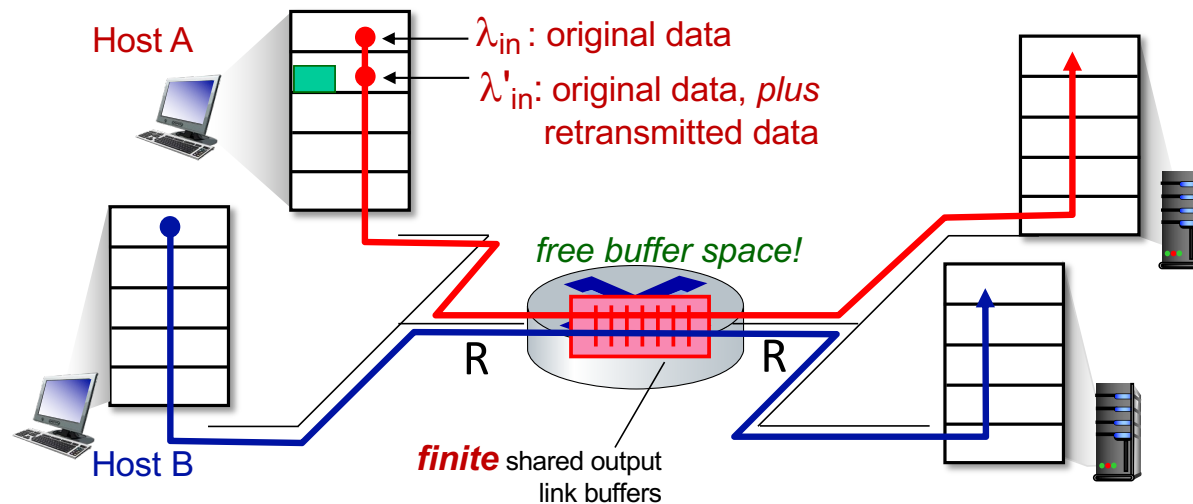
- packets can be lost (dropped at router) due to full buffers
- sender knows when packet has been dropped: only resends if packet *known* to be lost



Causes/costs of congestion: scenario 1

Idealization: *some* perfect knowledge

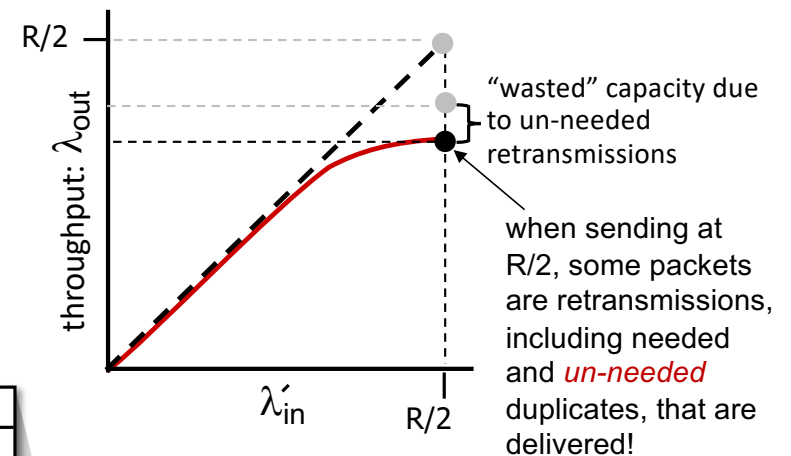
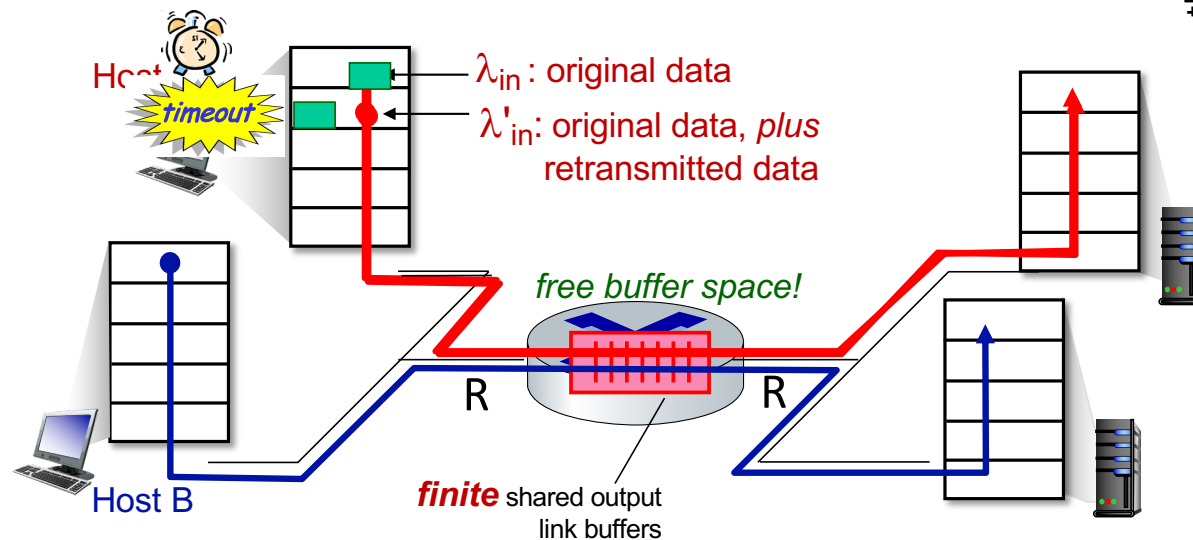
- packets can be lost (dropped at router) due to full buffers
- sender knows when packet has been dropped: only resends if packet *known* to be lost



Causes/costs of congestion: scenario 2

Realistic scenario: *un-needed duplicates*

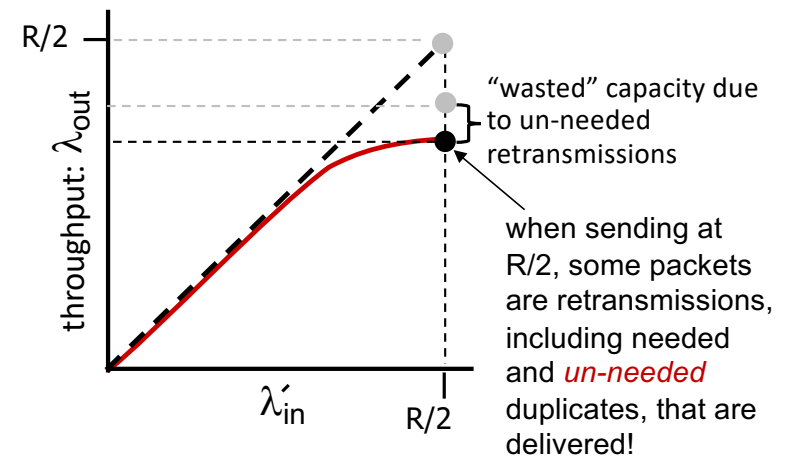
- packets can be lost, dropped at router due to full buffers – requiring retransmissions
- but sender times can time out prematurely, sending *two* copies, *both* of which are delivered



Causes/costs of congestion: scenario 2

Realistic scenario: *un-needed duplicates*

- packets can be lost, dropped at router due to full buffers – requiring retransmissions
- but sender times can time out prematurely, sending *two* copies, *both* of which are delivered



"costs" of congestion:

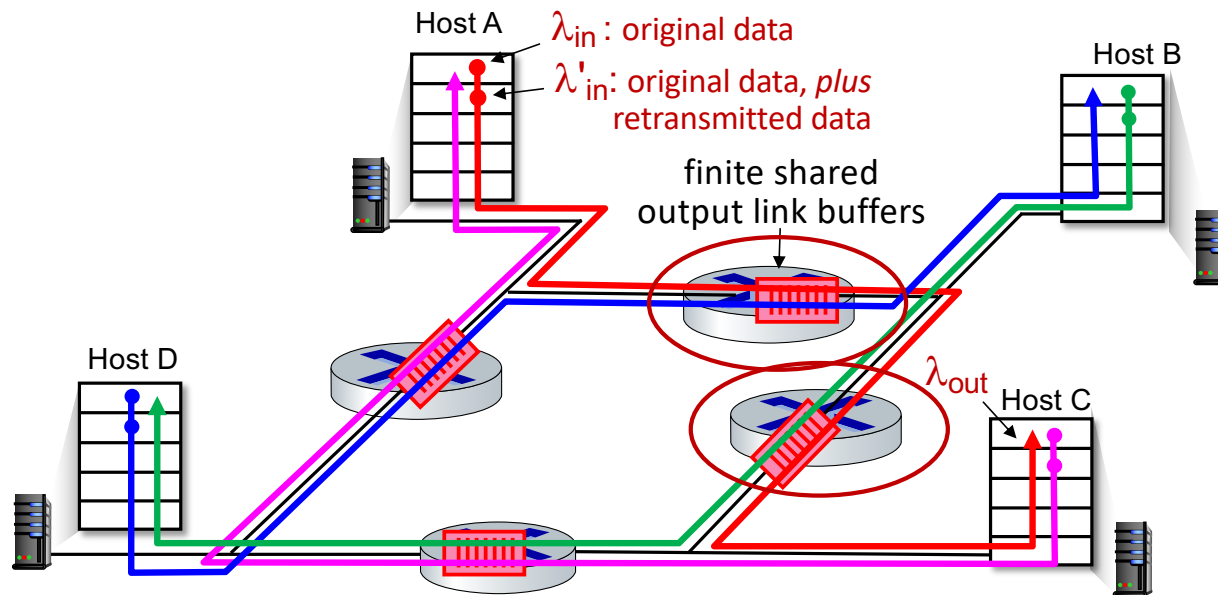
- more work (retransmission) for given receiver throughput
- unneeded retransmissions: link carries multiple copies of a packet
 - decreasing maximum achievable throughput

Causes/costs of congestion: scenario 3

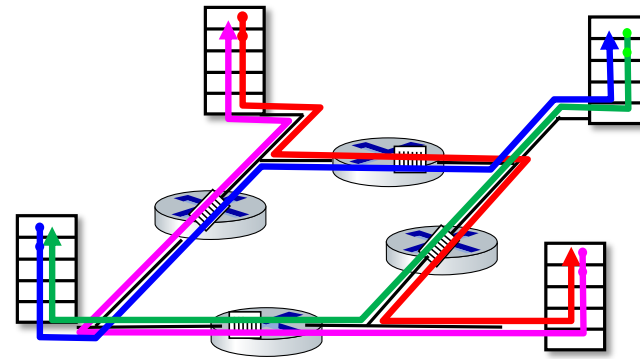
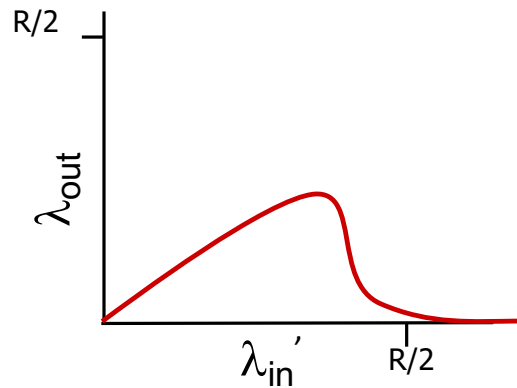
- *four* senders
- *multi-hop* paths
- timeout/retransmit

Q: what happens as λ_{in} and λ_{in}' increase ?

A: as red λ_{in}' increases, all arriving blue pkts at upper queue are dropped, blue throughput $\rightarrow 0$



Causes/costs of congestion: scenario 3

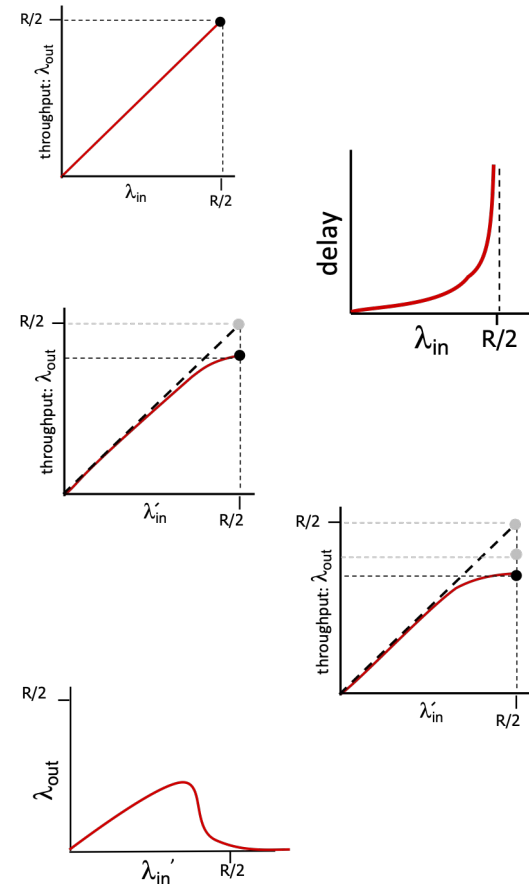


another “cost” of congestion:

- when packet dropped, any upstream transmission capacity and buffering used for that packet was wasted!

Causes/costs of congestion: insights

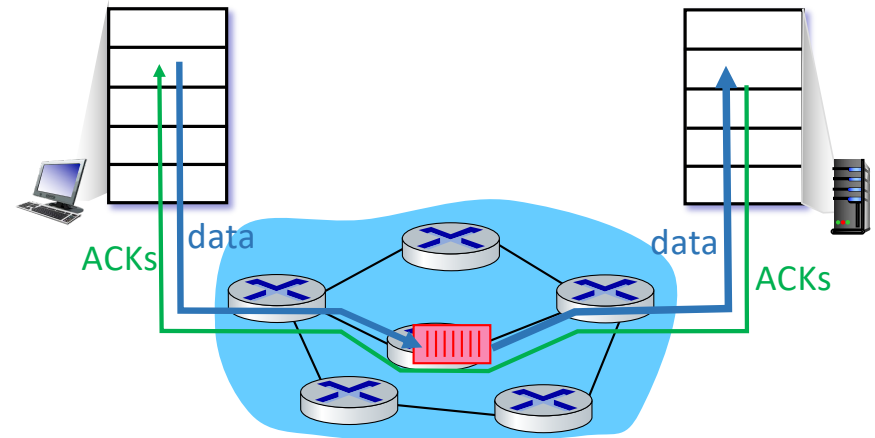
- throughput can never exceed capacity
- delay increases as capacity approached
- loss/retransmission decreases effective throughput
- un-needed duplicates further decreases effective throughput
- upstream transmission capacity / buffering wasted for packets lost downstream



Approaches towards congestion control

End-end congestion control:

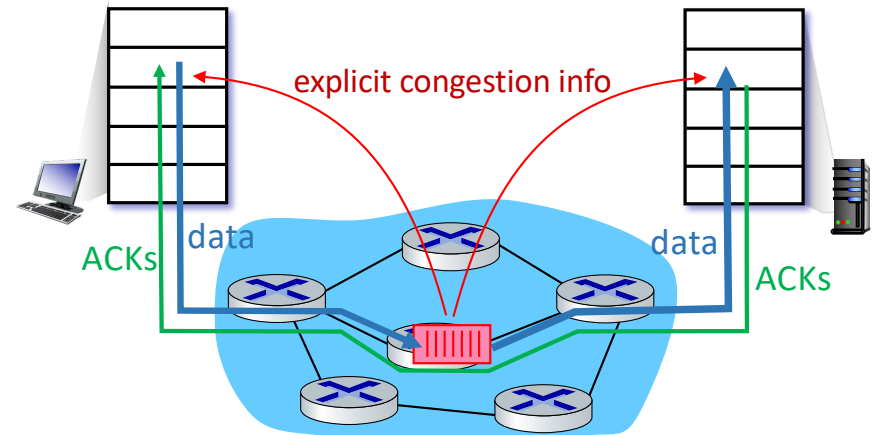
- no explicit feedback from network
- congestion *inferred* from observed loss, delay
- approach taken by TCP



Approaches towards congestion control

Network-assisted congestion control:

- routers provide *direct* feedback to sending/receiving hosts with flows passing through congested router
- may indicate congestion level or explicitly set sending rate
- TCP ECN, ATM, DECbit protocols



Transport layer: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- **TCP congestion control**
- Evolution of transport-layer functionality

How Does the Sender Know?

- Observe packet losses

- Timeout
- Duplicated ACKs



- Monitor packet delays

- RTT estimate



How Should the Sender Respond?

- What's your idea?
- TCP's idea
 - **Estimate the available capacity** → know how many packets to have in fly (**congestion window**)
 - Remember the sliding window?
 - **Adapt the congestion window** → decrease upon packet loss, and increase upon success

All These Windows...

- Congestion Window: **CWND**
 - # of **bytes** sent without overflowing routers
 - Computed by the sender using congestion control algorithm
- Flow control window: **AdvertisedWindow (RWND)**
 - # of **bytes** sent without overflowing receiver's buffers
 - Determined by the receiver and reported to the sender
- TCP window = **min**{**CWND**, **RWND**}
 - Assume for this lecture that $RWND \gg CWND$

Estimating Bandwidth with 'Slow Start'

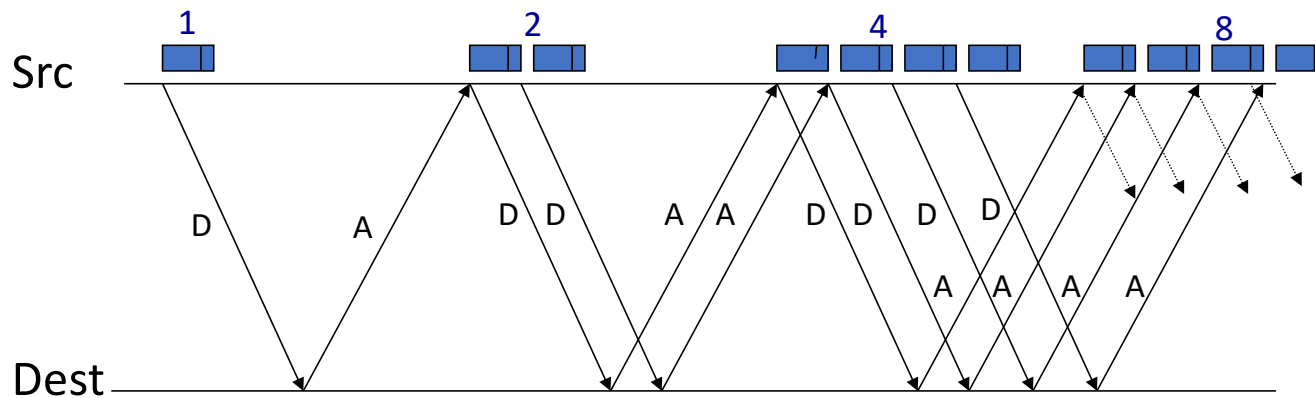
- Essentially a “poking” idea
- Sender starts with a slow rate
 - Initial **cwnd** = 1 MSS (Maximal segment size)
- Then increases **exponentially** until first packet loss
 - **Double cwnd** for each RTT without loss
 - done by incrementing **cwnd** for every ACK received



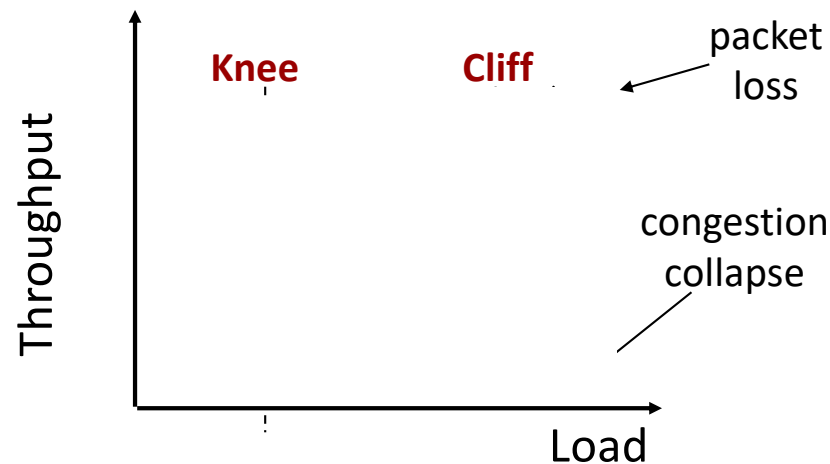
Not really a slow start!

Slow Start Example

- Double CWND per round-trip time



Eventually will hit the capacity...



- **Knee:** point after which throughput increases slowly
- **Cliff:** point after which throughput starts to drop to zero (congestion collapse)

How to adapt congestion window size?

- Pay more penalty for over-sized window than under-sized window
 - Over-sized window: packet loss and retransmissions
 - Under-sized window: lower throughput
- **Additive increase, multiplicative decrease (AIMD)**
 - Packet success: increase the congestion window linearly
 - Packet loss: **divide** congestion window by **half**

TCP congestion control: AIMD

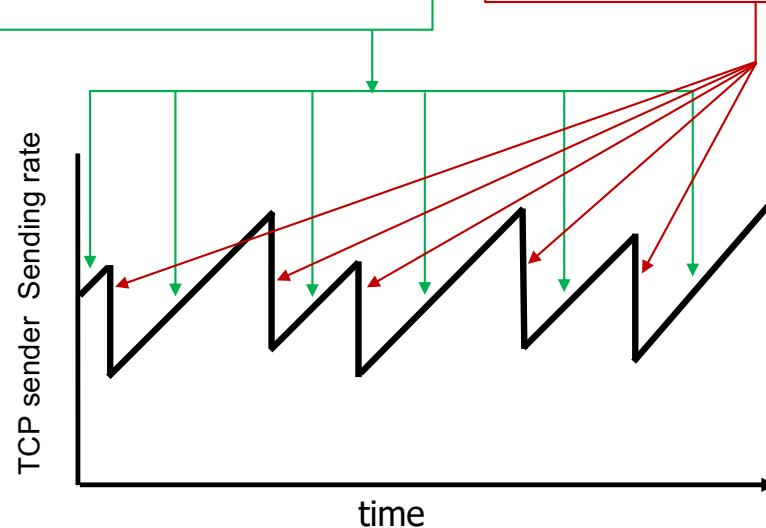
- *approach*: senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event

Additive Increase

increase sending rate by 1 maximum segment size every RTT until loss detected

Multiplicative Decrease

cut sending rate in half at each loss event



AIMD sawtooth
behavior: *probing*
for bandwidth

TCP AIMD: more

Multiplicative decrease detail: sending rate is

- Cut in half on loss detected by triple duplicate ACK (TCP Reno)
- Cut to 1 MSS (maximum segment size) when loss detected by timeout (TCP Tahoe)

Why AIMD?

- AIMD – a distributed, asynchronous algorithm – has been shown to:
 - optimize congested flow rates network wide!
 - have desirable stability properties

QUIZ: File distribution time

100 clients would like to download a popular file (50MB). Each client has 1Mbps download capacity and 0.5Mbps upload capacity.

- How much time would it take if they are downloading from a server in a client-server architecture, assuming the server supports 10 concurrent downloads?
- How much time would it take if they form a BitTorrent P2P system, assuming perfect distribution? Approximation is fine.